Software Engineering
Assignment-6
TYPE OF QUESTION: MCQ/MSQ

Number of questions: **10** **Total mark: 10 X 1 = 10**

QUESTION 1:

Which of the following statements correctly differentiate function-oriented design from object-oriented design?

a. Function-oriented design generally follows a bottom-up approach
b. Structured analysis and structured design are associated with function-oriented design
c. UML is commonly used in object-oriented design
d. Function-oriented design primarily focuses on objects and their interactions
e. High-level Object-oriented design primarily focuses on designing algorithms and data structures for classes.

Correct Answer: b. Structured analysis and structured design are associated with function-oriented design.
c. UML is commonly used in object-oriented design

Detailed Solution:

Structured analysis/design are indeed associated with function-oriented design (b), and UML is the standard notation for object-oriented design (c). Option a is false because function-oriented design follows a top-down approach, and option d is false because it describes OOD, not function-oriented design. Option e ("High-level Object-oriented design primarily focuses on designing algorithms and data structures for classes") is incorrect because high-level OOD focuses on class identification, relationships, and architecture—not on detailed algorithms and data structures, which belong to detailed design.

QUESTION 2:

Which of the following statements correctly describe structured analysis and structured design?

a. Structured analysis follows a bottom-up decomposition approach
b. Structured analysis decomposes high-level functions into more detailed functions
c. Structured design maps detailed functions onto module structures
d. Structured analysis primarily focuses on identifying objects and classes
e. Structured design helps organize software into hierarchical modules
f. Structured design is closely associated with procedural software development

Correct Answer: b. Structured analysis decomposes high-level functions into more detailed functions
c. Structured design maps detailed functions onto module structures
e. Structured design helps organize software into hierarchical modules
f. Structured design is closely associated with procedural software development

Detailed Solution:

Structured analysis uses top-down functional decomposition (b), and structured design maps those functions into module structures (c), organizing them hierarchically (e)—both are closely tied to procedural development (f). Structured analysis follows a top-down (not bottom-up) approach (a), and it focuses on functions and data flow, not objects and classes (d)—that is object-oriented analysis.

QUESTION 3:

A Data Flow Diagram (DFD) primarily represents which of the following aspects of a software system?

a. Decision logic used in conditional statements
b. Flow of data among processes, data stores, and external entities
c. Sequence in which program statements are executed
d. Transformation of input data into output data through processing steps
e. Movement of data between external entities, processes, and data stores
f. Invocation order of program functions during execution

Correct Answer: b. Flow of data among processes, data stores, and external entities
d. Transformation of input data into output data through processing steps
e. Movement of data between external entities, processes, and data stores

Detailed Solution:

A DFD primarily represents the flow of data among processes, data stores, and external entities (b), the transformation of input data into output data through processing steps (d), and the movement of data between those elements (e). It does not show decision logic (a), execution sequence (c), or function invocation order (f)—those are represented in flowcharts, structure charts, or sequence diagrams.

QUESTION 4:

Which of the following statements are NOT true regarding a Data Flow Diagram (DFD)?

a. A DFD represents the procedural logic of program code
b. A DFD is usually developed using hierarchical decomposition
c. A DFD explicitly represents decision-making conditions and control flow
d. A DFD represents how data is transformed through processes
e. A DFD is normally constructed using a top-down approach
f. A DFD directly represents the module call structure of a software system

Correct Answer: a. A DFD represents the procedural logic of program code
c. A DFD explicitly represents decision-making conditions and control flow
f. A DFD directly represents the module call structure of a software system

Detailed Solution:

DFDs do not represent procedural logic (a), decision-making conditions or control flow (c)—those are shown in flowcharts or structure charts—nor do they represent module call structure (f), which is shown in structure charts. DFDs are developed using hierarchical/top-down decomposition (b, e) and do represent data transformation through processes (d), so those statements are true.

QUESTION 5:

Which of the following statements correctly describe functional decomposition in structured analysis and design?

a. Functional decomposition involves recursive calls among functions
b. High-level data may also be decomposed into more detailed data representations
c. High-level functions are successively decomposed into more detailed functions
d. Functional decomposition focuses only on control flow between statements
e. Decomposition helps manage complexity by dividing a system into smaller parts
f. Functional decomposition is unrelated to modular software design

Correct Answer: b. High-level data may also be decomposed into more detailed data representations
c. High-level functions are successively decomposed into more detailed functions
e. Decomposition helps manage complexity by dividing a system into smaller parts

Detailed Solution:

Functional decomposition breaks high-level functions into more detailed ones (c) and may also decompose data into finer representations (b), helping manage complexity by dividing the system into smaller parts (e). It does not involve recursive calls (a) (that's a programming technique), focus only on control flow (d) (it's about function hierarchy), nor is it unrelated to modular design (f)—it directly leads to modular structures.

QUESTION 6:

Which of the following statements correctly describe balancing in Data Flow Diagrams (DFDs)?

a. Balancing primarily ensures preservation of program control logic across DFD levels
b. The input data flows to a process must be preserved when that process is decomposed into a lower-level DFD
c. The output data flows from a process must be preserved in its decomposed lower-level DFD
d. Balancing ensures consistency of data flow between different levels of DFDs
e. A lower-level DFD may introduce completely new external input flows not present in the parent process
f. Data entering and leaving a parent process should correspond to the combined input and output flows of its child processes

Correct Answer: b. The input data flows to a process must be preserved when that process is decomposed into a lower-level DFD
c. The output data flows from a process must be preserved in its decomposed lower-level DFD
d. Balancing ensures consistency of data flow between different levels of DFDs
f. Data entering and leaving a parent process should correspond to the combined input and output flows of its child processes

Detailed Solution:

Balancing ensures that input/output data flows of a parent process are preserved when decomposed into lower-level DFDs (b, c), maintaining consistency across levels (d)—the net effect of child processes must match the parent's flows (f). It does not preserve control logic (a) (DFDs don't show control), and new external inputs cannot be introduced (e)—that would violate balancing.

QUESTION 7:

Which of the following activities are typically performed while constructing a Level-1 Data Flow Diagram (DFD)?

a. Representing each high-level function as a separate process (bubble)
b. Representing data inputs to each high-level function
c. Representing data outputs produced by each high-level function
d. Writing detailed program code for each process
e. Identifying data stores and data movement between processes
f. Specifying the exact sequence of statement execution within each module

Correct Answer: a. Representing each high-level function as a separate process (bubble)
b. Representing data inputs to each high-level function
c. Representing data outputs produced by each high-level function
e. Identifying data stores and data movement between processes

Detailed Solution:

Constructing a Level-1 DFD involves representing each high-level function as a separate process (bubble), showing data inputs and outputs for each, and identifying data stores and data movement between processes. It does not involve writing code (d) or specifying execution sequence (f)—those are implementation and control-flow details, not DFD modeling.

QUESTION 8:

For a given software system, a Data Flow Diagram (DFD) is primarily used to represent which of the following?

a. Flow and transformation of data within the system
b. Flow of control between program statements
c. Movement of data between processes, data stores, and external entities
d. Sequence of function calls in a program
e. Processing activities that transform input data into output data
f. Module hierarchy of the software system

Correct Answer: a. Flow and transformation of data within the system
c. Movement of data between processes, data stores, and external entities
e. Processing activities that transform input data into output data

Detailed Solution:

A DFD primarily represents the flow and transformation of data (a), the movement of data between processes, data stores, and external entities (c), and the processing activities that transform inputs into outputs (e). It does not represent control flow (b), function call sequences (d), or module hierarchy (f)—those are shown in flowcharts, sequence diagrams, or structure charts.

QUESTION 9:

Which of the following statements correctly describe structured analysis and structured design?

a. Structured analysis primarily concentrates on analyzing syntax and structure of a program
b. Structured analysis focuses on capturing the system from the user's perspective
c. Structured design transforms the analysis model into a form suitable for implementation
d. Structured design helps derive a modular implementation structure
e. Structured analysis is concerned with understanding system requirements and functionality
f. Structured design ignores implementation considerations completely

Correct Answer: b. Structured analysis focuses on capturing the system from the user's perspective
c. Structured design transforms the analysis model into a form suitable for implementation
d. Structured design helps derive a modular implementation structure
e. Structured analysis is concerned with understanding system requirements and functionality

Detailed Solution:

Structured analysis captures system functions and data from the user's perspective (b) to understand requirements (e), while structured design transforms this analysis model into a modular implementation structure (c, d) suitable for coding. Structured analysis does not focus on program syntax (a) (that's code-level), and structured design does not ignore implementation (f)—it explicitly considers it.

QUESTION 10:

Which of the following statements correctly describe a data dictionary in structured analysis?

a. All data items appearing in a data dictionary map to variables in the program implementation
b. A data dictionary describes composite data items in terms of their component data elements
c. A data dictionary primarily represents control flow between modules
d. A DFD is commonly accompanied by a data dictionary
e. A data dictionary includes the names and purposes of data items
f. A data dictionary specifies the sequence of execution of program statements

Correct Answer: b. A data dictionary describes composite data items in terms of their component data elements
d. A DFD is commonly accompanied by a data dictionary
e. A data dictionary includes the names and purposes of data items

Detailed Solution:

A data dictionary describes composite data items by breaking them into component elements (b), is commonly used alongside DFDs (d), and includes names and purposes of data items (e). It does not require every data item to map directly to program variables (a), does not represent control flow (c), and does not specify execution sequence (f)—those are implementation and control-flow

---

******THE END******